



Southeast University

Department of Computer Science and Engineering (CSE)

School of Sciences and Engineering

Semester: (Summer, Year: 2026)

LAB REPORT NO: 06

Course Title: Operating Systems Lab

Course Code: CSE362.2

Batch: 65

**Lab Experiment Name: Implementing Round Robin algorithm using C, C++
or bash scripting.**

Student Details

Name		ID
1.	Md. Mahmud Hossain	2023200000799

Course Teacher's Name : Md. Ismail

Designation : Lecturer

Submission Date : 31-08-26

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

Introduction:

Round Robin (RR) is a preemptive CPU scheduling algorithm designed specifically for time-sharing systems. Each process in the ready queue is assigned a fixed time unit called a Time Quantum (or time slice). The CPU executes a process for up to one time quantum. If the process does not complete within this duration, it is preempted and added to the back of the ready queue, allowing the next waiting process to execute. This round-robin structure ensures fair allocation of CPU resources and prevents process starvation.

Objectives:

- To understand the working principles of preemptive Round Robin CPU scheduling.
- To implement Round Robin scheduling in C++ using a queue data structure.
- To dynamically process user inputs (Arrival Time, Burst Time, Time Quantum).
- To compute individual process metrics (CT , TAT , WT) and aggregate system performance (Average WT , Average TAT , Total Idle Time, and CPU Utilization).

Implementation:

Implementing Round Robin scheduling algorithm

C++ code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <iomanip>
```

```
using namespace std;
```

```
struct Process {
    int pid;
    int at;
    int bt;
    int rem_bt;
    int ct;
    int tat;
    int wt;
};
```

```
int main() {
    int n, quantum;
    cout << "Enter number of processes: ";
```

```

cin >> n;
cout << "Enter Time Quantum: ";
cin >> quantum;

vector<Process> p(n);
for (int i = 0; i < n; i++) {
    cout << "Process " << i + 1 << " - Enter PID, Arrival Time, Burst Time: ";
    cin >> p[i].pid >> p[i].at >> p[i].bt;
    p[i].rem_bt = p[i].bt;
}

// Sort processes by Arrival Time
sort(p.begin(), p.end(), [](const Process &a, const Process &b) {
    return a.at < b.at;
});

queue<int> ready_queue;
vector<bool> visited(n, false);

int current_time = 0;
int completed = 0;
int idle_time = 0;

// Account for CPU idle time if first process arrives after t = 0
if (p[0].at > 0) {
    idle_time += p[0].at;
    current_time = p[0].at;
}

ready_queue.push(0);
visited[0] = true;

while (completed < n) {
    if (ready_queue.empty()) {
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                idle_time += (p[i].at - current_time);
                current_time = p[i].at;
                ready_queue.push(i);
                visited[i] = true;
            }
        }
    }
}

```

```

        break;
    }
}

int idx = ready_queue.front();
ready_queue.pop();

int slice = min(p[idx].rem_bt, quantum);
p[idx].rem_bt -= slice;
current_time += slice;

// Push processes that arrived during this time slice
for (int i = 0; i < n; i++) {
    if (!visited[i] && p[i].at <= current_time) {
        ready_queue.push(i);
        visited[i] = true;
    }
}

// If current process has remaining burst time, re-queue it
if (p[idx].rem_bt > 0) {
    ready_queue.push(idx);
} else {
    p[idx].ct = current_time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    completed++;
}
}

// Output table and averages
float total_tat = 0, total_wt = 0;
cout << "\n===== ROUND ROBIN SCHEDULING =====\n";
cout << "PID\tAT\tBT\tCT\tTAT\tWT\n";
cout << "-----\n";
for (int i = 0; i < n; i++) {
    total_tat += p[i].tat;
    total_wt += p[i].wt;
    cout << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt << "\t"

```

```

        << p[i].ct << "\t" << p[i].tat << "\t" << p[i].wt << "\n";
    }

    double cpu_utilization = ((double)(current_time - idle_time) / current_time) * 100;

    cout << fixed << setprecision(2);
    cout << "\nAverage Waiting Time: " << total_wt / n;
    cout << "\nAverage Turnaround Time: " << total_tat / n;
    cout << "\nTotal Idle Time: " << idle_time << " units";
    cout << "\nCPU Utilization: " << cpu_utilization << "%\n";

    return 0;
}

```

Screenshot:

```

mahmud@mahmud-VMware-Virtual-Platform:~$ g++ rr.cpp -o rr
mahmud@mahmud-VMware-Virtual-Platform:~$ ./rr
Enter number of processes: 3
Enter Time Quantum: 2
Process 1 - Enter PID, Arrival Time, Burst Time: 1 0 5
Process 2 - Enter PID, Arrival Time, Burst Time: 2 1 3
Process 3 - Enter PID, Arrival Time, Burst Time: 3 2 1

===== ROUND ROBIN SCHEDULING =====
PID      AT      BT      CT      TAT      WT
-----
1         0       5       9       9       4
2         1       3       8       7       4
3         2       1       5       3       2

Average Waiting Time: 3.33
Average Turnaround Time: 6.33
Total Idle Time: 0 units
CPU Utilization: 100.00%
mahmud@mahmud-VMware-Virtual-Platform:~$

```

Conclusion:

In this experiment, the preemptive Round Robin CPU scheduling algorithm was successfully implemented in C++. By allocating CPU execution in time slices determined by the time quantum, every process was guaranteed responsive turnaround and fair processor access. Performance evaluation metrics including Average Waiting Time, Average Turnaround Time, Total Idle Time, and CPU Utilization were accurately calculated, highlighting how selecting an optimal time quantum plays a pivotal role in balancing system throughput and response time.